

BUSINESS ANALYTIC PROJECT

Business Analytics for Campus Cafe Sales and Profit Analysis

Student Name: Afsha Amjad Mujawar

Roll Number: T025

Class: B.Sc. Information Technology

Division: A

Subject: Applied Business Analytics

Academic Year: 2026–2027

College Name: SHETH L.U.J & SIR MV COLLAGE

Department Name: IT

Faculty Name: Ms. Chetana Tanawade

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Business Scenario / Domain Description
5. Dataset Description
6. Data Collection
7. Data Preprocessing
8. Exploratory Data Analysis
9. Descriptive Statistical Analysis
10. Distribution Analysis
11. Correlation Analysis
12. Regression Analysis
13. Business Intelligence Dashboard
14. Probability and Probability Distributions
15. Decision Making Under Uncertainty
16. Sampling and Sampling Distribution
17. Confidence Interval Estimation
18. Bootstrapping
19. Hypothesis Testing

20. A/B Testing
21. Time Series Analysis
22. Forecast Accuracy
23. Optimization Modeling
24. Simulation Modeling
25. ANOVA
26. Experimental Design
27. Factorial Experiment
28. Business Insights and Findings
29. Business Recommendations
30. Conclusion
31. Appendix

1. Introduction

Business Analytics is the process of using data, statistical methods, and analytical techniques to understand business performance and support better decision-making.

This project focuses on **Campus Cafe**, a cafe business that sells different food and beverage products through different outlets and regions.

The dataset contains information about orders, customers, products, categories, outlets, quantities, prices, discounts, sales, and profits.

Business Analytics can help Campus Cafe to:

- Identify high-performing products.
- Analyze sales and profit.
- Understand customer purchasing patterns.
- Compare outlets and regions.
- Identify relationships between business variables.
- Forecast future sales.
- Support pricing and inventory decisions.

The project follows the ABA requirements for EDA, statistics, visualization, correlation, regression, probability, forecasting, optimization, simulation, ANOVA, and experimental analysis. APPLIED BUSINESS PROJECT

2. Problem Statement

To analyze Campus Cafe sales and customer transaction data to identify sales and profit patterns, relationships between business variables, product and outlet performance, future sales trends, and suitable business strategies for improving business performance.

The project will use statistical and analytical techniques to support business decision-making.

3. Objectives

1. To collect and analyze Campus Cafe business data.
2. To perform exploratory data analysis.
3. To analyze sales, profit, quantity, price, and discount.
4. To identify relationships between numerical variables.
5. To compare products, categories, outlets, and regions.
6. To perform statistical hypothesis testing.
7. To forecast future sales.
8. To develop data-driven business recommendations.

4. Business Scenario / Domain Description

Business Name

Campus Cafe

Business Type

Food and beverage / cafe business.

Products/Services

The cafe offers different food and beverage products across different categories and outlets.

Target Customers

Students, faculty members, staff, and other customers visiting the cafe.

Business Operations

Campus Cafe receives customer orders through its outlets. Each transaction contains product, quantity, unit price, discount, sales, profit, location, and customer information.

Business Problems

The business needs to understand:

- Which products generate higher sales?
- Which products generate higher profit?
- Which outlets perform better?
- How discounts affect sales?
- What factors are related to profit?
- What future sales can be expected?

5. Dataset Description

This specifically requires the dataset name, source, number of records, columns, column descriptions, data types, numerical variables, categorical variables, and date/time variables. APPLIED BUSINESS PROJECT

Dataset Information:

Dataset Name: Campus Cafe Dataset

Number of Records: 500

Number of Columns: 17

File Type: CSV

Description of Columns:

Column	Description	Data Type	Variable Type
--------	-------------	-----------	---------------

Order_ID	Unique order identifier	Object	Categorical
Order_Date	Date of order	Datetime	Date
Customer_ID	Unique customer identifier	Object	Categorical
Customer_Name	Customer name	Object	Categorical
Product	Product purchased	Object	Categorical
Category	Product category	Object	Categorical
Brand	Brand associated with product	Object	Categorical
Outlet	Campus cafe outlet	Object	Categorical
Region	Mumbai region of outlet	Object	Categorical
City	City of outlet	Object	Categorical
Latitude	Outlet latitude	Float64	Numerical
Longitude	Outlet longitude	Float64	Numerical
Quantity	Units ordered	Int64	Numerical
Unit_Price	Price per unit	Int64	Numerical
Discount	Discount applied	Float64	Numerical
Sales	Sales value after discount	Float64	Numerical
Profit	Profit from order	Float64	Numerical

Numerical Variables

Latitude, Longitude, Quantity, Unit_Price, Discount, Sales, Profit

Categorical Variables

Order_ID, Customer_ID, Customer_Name, Product, Category, Brand, Outlet, Region, City

Date Variable

Order_Date

6. Data Collection

The Campus Cafe dataset was provided in CSV format and contains 500 transaction records. The dataset contains customer, product, outlet, sales, pricing, discount, and profit information.

Code:

```
import pandas as pd
df = pd.read_csv("Campus Cafe Dataset.csv")
print("Number of records:", len(df))
print("Number of columns:", len(df.columns))
df.head()
```

Output :

Number of records: 500 Number of columns: 17											
	Order_ID	Order_Date	Customer_ID	Customer_Name	Product	Category	Brand	Outlet	Region	City	Latitude
0	ORD0398	2026-01-01	CUST049	Sana	Iced Lemon Tea	Beverage	BrewRush	Sports Ground Stall	Western Mumbai	Mumbai	19.1197
1	ORD0331	2026-01-03	CUST028	Arjun	Masala Chai	Beverage	BrewRush	Science Block Café	Central Mumbai	Mumbai	19.0178
2	ORD0102	2026-01-05	CUST039	Rahul	Cheese Roll	Bakery	SnackLab	Library Café	South Mumbai	Mumbai	18.9988
3	ORD0261	2026-01-05	CUST004	Vivaan	Veg Puff	Bakery	SnackLab	Science Block Café	Central Mumbai	Mumbai	19.0178
4	ORD0445	2026-01-05	CUST010	Sara	Fruit Bowl	Healthy	FreshFuel	Tech Lab Café	Western Mumbai	Mumbai	19.1075

7. Data Preprocessing:

Data preprocessing is the process of **cleaning and preparing the dataset** before analysis by checking missing values, duplicates, incorrect data types, and invalid values.

Code:

1. Import Libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Campus Cafe Dataset.csv")
```

2. Check Dataset Shape:

```
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

3. Check Column Names

```
print(df.columns)
```

3. Check Data Types:

```
print(df.dtypes)
```

4. Convert Date:

```
df['Order_Date'] = pd.to_datetime(df['Order_Date'])
print(df['Order_Date'].dtype)
```

5. Missing Values:

```
print(df.isnull().sum())
```

6. Duplicate Record:

```
print("Duplicate records:", df.duplicated().sum())
```

7. Statistical Summary:

```
print(df.describe())
```

8. Check Unique Values:

```
for col in df.select_dtypes(include='object').columns:
    print("\n", col)
```

9. Check Invalid Numerical Values:

```
numeric_cols = [
    'Quantity',
    'Unit_Price',
    'Discount',
    'Sales',
    'Profit'
]
for col in numeric_cols:
    print(col, "minimum =", df[col].min())
```

Output:

```
... Rows: 500
Columns: 17
Index(['Order_ID', 'Order_Date', 'Customer_ID', 'Customer_Name', 'Product',
      'Category', 'Brand', 'Outlet', 'Region', 'City', 'Latitude',
      'Longitude', 'Quantity', 'Unit_Price', 'Discount', 'Sales', 'Profit'],
      dtype='str')
Order_ID      str
Order_Date    str
Customer_ID    str
Customer_Name  str
Product        str
Category       str
Brand          str
Outlet         str
Region         str
City           str
Latitude       float64
Longitude      float64
Quantity       int64
Unit_Price     int64
Discount       float64
Sales          float64
Profit         float64
dtype: object
datetime64[us]
...
Unit_Price minimum = 25
Discount minimum = 0.0
Sales minimum = 21.25
Profit minimum = 5.5
```

8. Exploratory Data Analysis (EDA):

Exploratory Data Analysis (EDA) is the process of examining and understanding the dataset using statistics and visualizations.

It helps identify **patterns, trends, relationships, missing values, and unusual data**.

In this project, EDA is performed using **bar charts, histograms, box plots, pie charts, and scatter plots**.

Code:

A. Basic of EDA:

1. Load Dataset

```
df = pd.read_csv("Campus Cafe Dataset.csv")
print("Dataset loaded successfully!")
```

2. Display First 5 Records

```
print("\nFIRST 5 RECORDS:")
print(df.head())
```

3. Display Last 5 Records

```
print("\nLAST 5 RECORDS:")
print(df.tail())
```

4. Dataset Shape

```
print("\nDATASET SHAPE:")
print(df.shape)
```

```
print("Number of Rows:", df.shape[0])
```

```
print("Number of Columns:", df.shape[1])
```

```

# 5. Column Names
print("\nCOLUMN NAMES:")
print(df.columns.tolist())

# 6. Data Types
print("\nDATA TYPES:")
print(df.dtypes)

# 7. Missing Values
print("\nMISSING VALUES:")
print(df.isnull().sum())

# 8. Total Missing Values
print("\nTOTAL MISSING VALUES:")
print(df.isnull().sum().sum())

# 9. Duplicate Records
print("\nNUMBER OF DUPLICATE RECORDS:")
print(df.duplicated().sum())

# 10. Unique Values
print("\nUNIQUE VALUES IN EACH COLUMN:")
print(df.nunique())

# 11. Basic Statistical Summary
print("\nSTATISTICAL SUMMARY:")
print(df.describe())

# 12. Numerical Columns
print("\nNUMERICAL COLUMNS:")
print(df.select_dtypes(include=np.number).columns.tolist())

# 13. Categorical Columns
print("\nCATEGORICAL COLUMNS:")
print(df.select_dtypes(include="object").columns.tolist())

```

Output:

```

Dataset loaded successfully!

FIRST 5 RECORDS:
  Order_ID  Order_Date  Customer_ID  Customer_Name  Product  Category \
0  ORD0398  2026-01-01  CUST049      Sana  Iced Lemon Tea  Beverage
1  ORD0331  2026-01-03  CUST028      Arjun  Masala Chai  Beverage
2  ORD0102  2026-01-05  CUST039      Rahul  Cheese Roll  Bakery
3  ORD0261  2026-01-05  CUST004      Vivaan  Veg Puff  Bakery
4  ORD0445  2026-01-05  CUST010      Sara  Fruit Bowl  Healthy

  Brand  Outlet  Region  City  Latitude \
0  BrewRush  Sports Ground Stall  Western Mumbai  Mumbai  19.1197
1  BrewRush  Science Block Café  Central Mumbai  Mumbai  19.0178
2  SnackLab  Library Café  South Mumbai  Mumbai  18.9988
3  SnackLab  Science Block Café  Central Mumbai  Mumbai  19.0178
4  FreshFuel  Tech Lab Café  Western Mumbai  Mumbai  19.1075

  Longitude  Quantity  Unit_Price  Discount  Sales  Profit
0  72.8468      3      40      0.05  114.0  42.00
1  72.8560      4      25      0.00  100.0  45.00
2  72.8258      1      45      0.10   40.5  11.25
3  72.8560      2      25      0.10   45.0  11.00
4  72.8370      2      65      0.00  130.0  39.00

LAST 5 RECORDS:
...
['Latitude', 'Longitude', 'Quantity', 'Unit_Price', 'Discount', 'Sales', 'Profit']

CATEGORICAL COLUMNS:
['Order_ID', 'Order_Date', 'Customer_ID', 'Customer_Name', 'Product', 'Category', 'Brand']
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
C:\Users\HAMITZA\AppData\Local\Temp\ipykernel_24548\2793304392.py:105: Pandas4Warning: For

```

B. EDA Charts

CHART 1: BAR CHART - Sales by Category

```
category_sales = df.groupby("Category")["Sales"].sum().sort_values(ascending=False)
plt.figure(figsize=(8,5))
category_sales.plot(kind="bar")
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

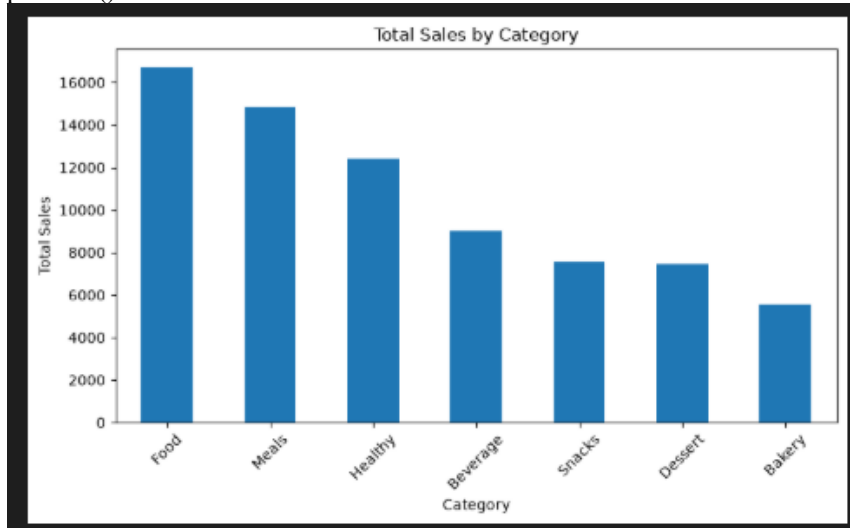


CHART 2: BAR CHART - Profit by Region

```
region_profit = df.groupby("Region")["Profit"].sum().sort_values(ascending=False)
```

```
plt.figure(figsize=(8,5))
region_profit.plot(kind="bar")
plt.title("Total Profit by Region")
plt.xlabel("Region")
plt.ylabel("Total Profit")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

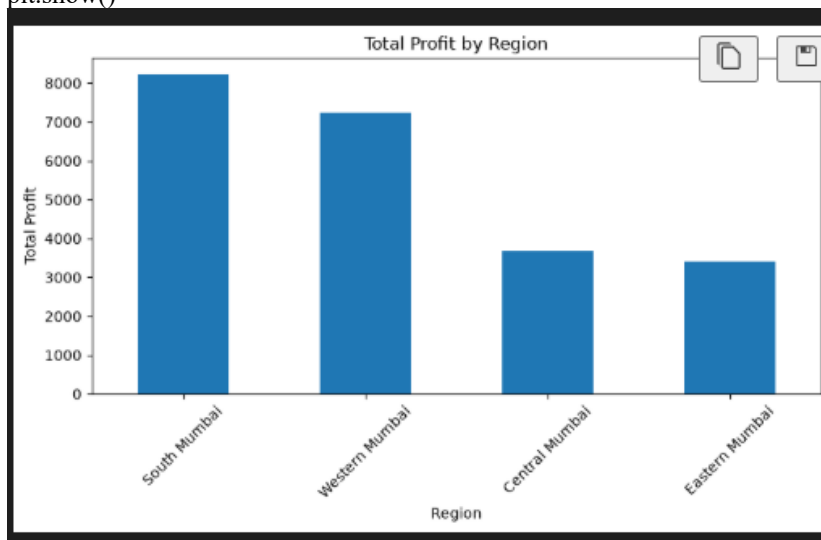


CHART 3: HISTOGRAM - Sales Distribution

```
plt.figure(figsize=(8,5))
plt.hist(df["Sales"], bins=10, edgecolor="black")
```

```
plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

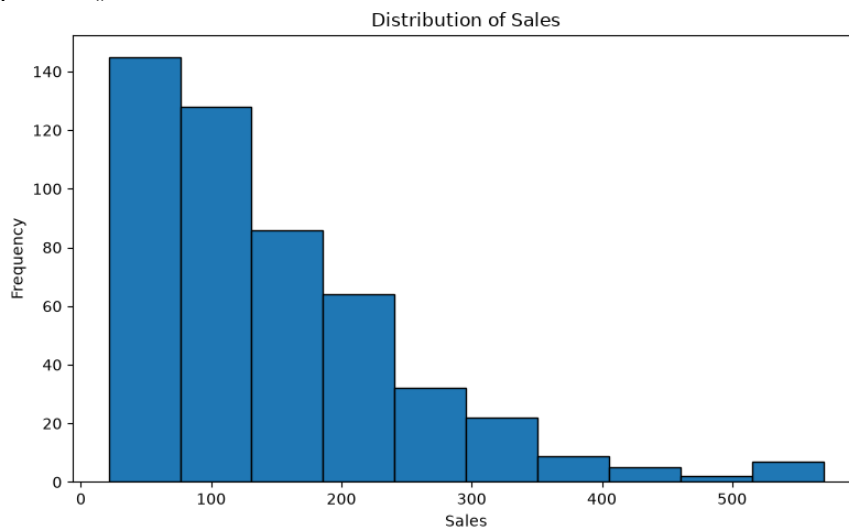


CHART 4: HISTOGRAM - Profit Distribution

```
plt.figure(figsize=(8,5))
plt.hist(df["Profit"], bins=10, edgecolor="black")
plt.title("Distribution of Profit")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```

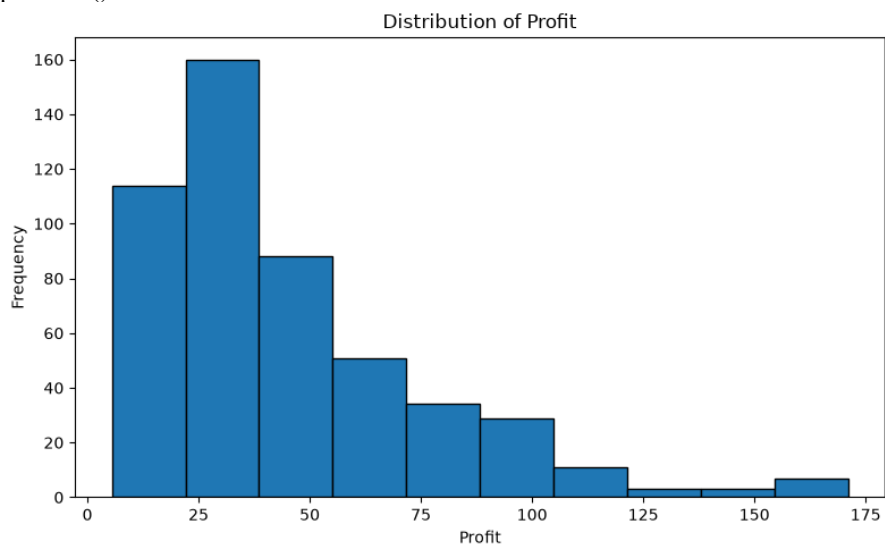


CHART 5: BOX PLOT - Sales

```
plt.figure(figsize=(8,5))
plt.boxplot(df["Sales"].dropna())
plt.title("Box Plot of Sales")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()
```

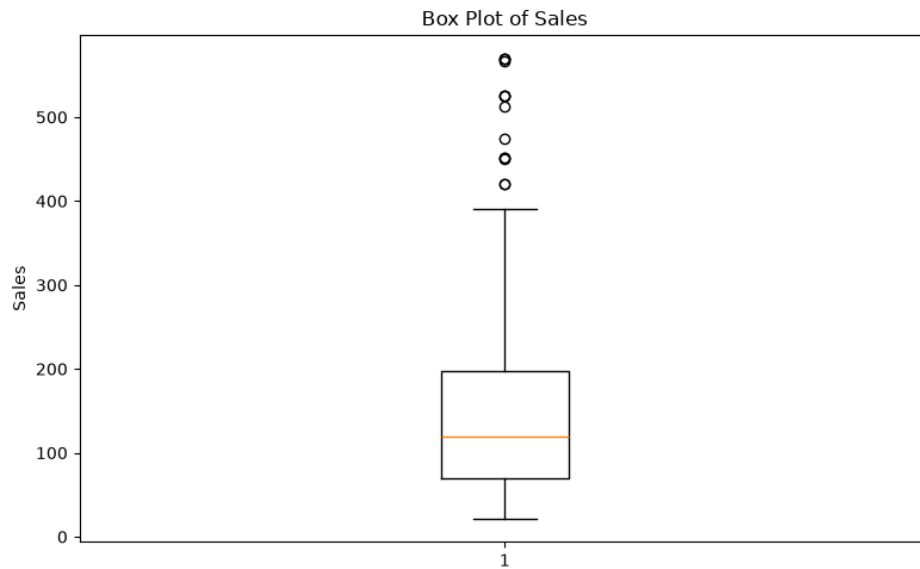


CHART 6: PIE CHART - Sales by Category

```
category_sales = df.groupby("Category")["Sales"].sum()
```

```
plt.figure(figsize=(7,7))
plt.pie(
    category_sales,
    labels=category_sales.index,
    autopct="%1.1f%%",
    startangle=90
)
plt.title("Sales Distribution by Category")
plt.show()
```

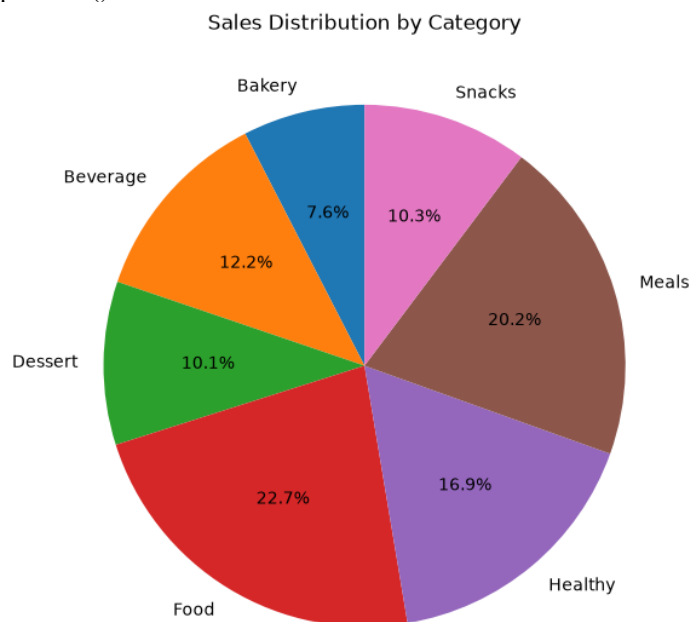


CHART 7: SCATTER PLOT - Sales vs Profit

```
plt.figure(figsize=(8,5))
plt.scatter(df["Sales"], df["Profit"])
plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
```

```
plt.tight_layout()
plt.show()
```



CHART 8: SALES BY PRODUCT

```
product_sales = df.groupby("Product")["Sales"].sum().sort_values(ascending=False).head(10)
```

```
plt.figure(figsize=(10,5))
product_sales.plot(kind="bar")
plt.title("Top 10 Products by Sales")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

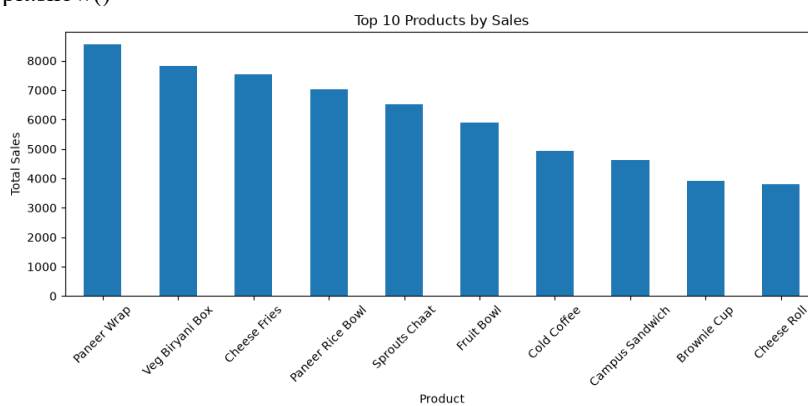


CHART 9: QUANTITY BY CATEGORY

```
category_quantity = df.groupby("Category")["Quantity"].sum().sort_values(ascending=False)
```

```
plt.figure(figsize=(8,5))
category_quantity.plot(kind="bar")
plt.title("Total Quantity Sold by Category")
plt.xlabel("Category")
plt.ylabel("Quantity")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

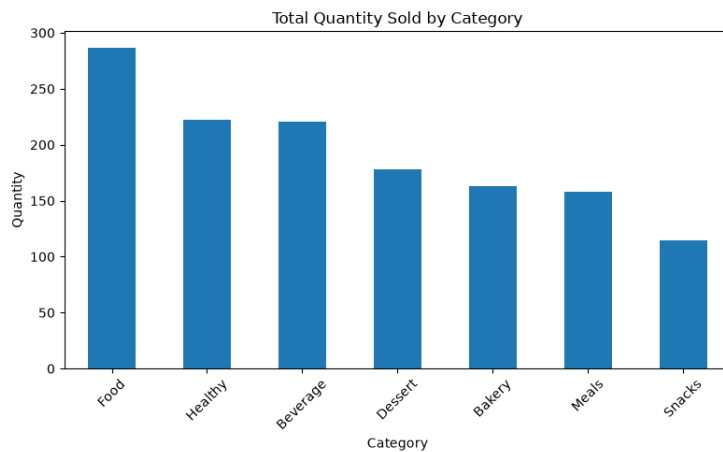
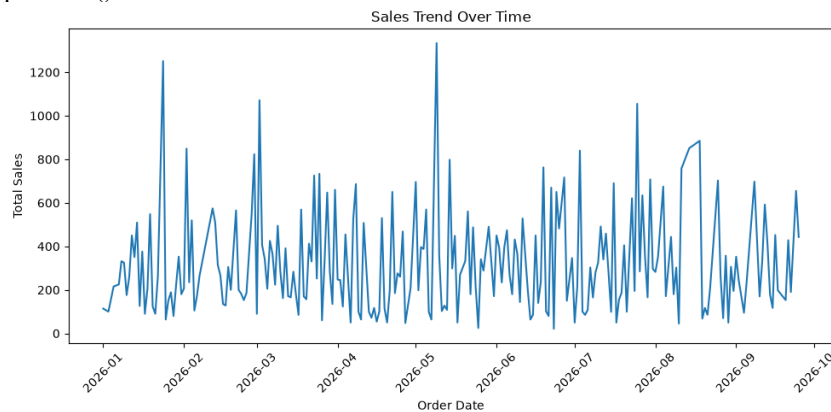


CHART 10: SALES TREND OVER TIME

```
df["Order_Date"] = pd.to_datetime(df["Order_Date"])
```

```
daily_sales = df.groupby("Order_Date")["Sales"].sum()
```

```
plt.figure(figsize=(10,5))
plt.plot(daily_sales.index, daily_sales.values)
plt.title("Sales Trend Over Time")
plt.xlabel("Order Date")
plt.ylabel("Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



14. Correlation Matrix

```
numerical_data = df.select_dtypes(include=np.number)
```

```
print("\nCORRELATION MATRIX:")
print(numerical_data.corr())
```

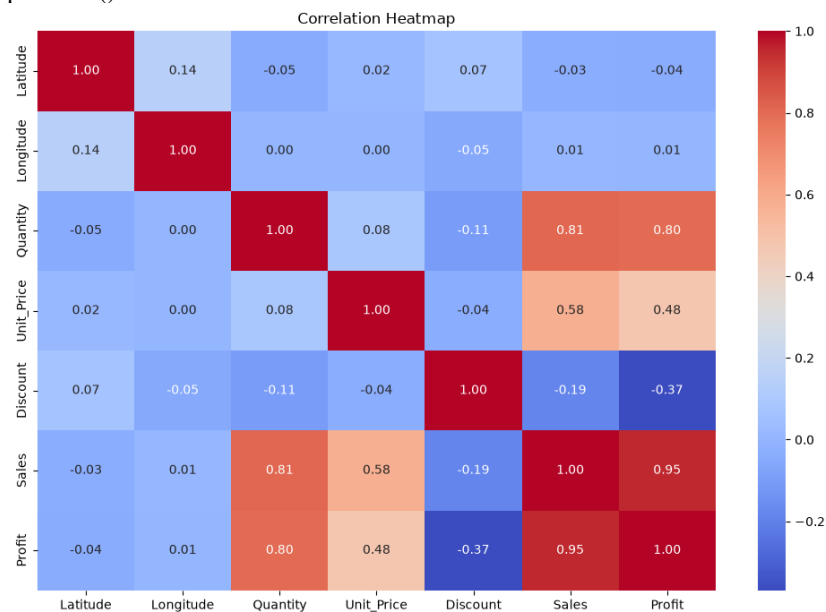
CORRELATION MATRIX:			
	Latitude	Longitude	Quantity
Latitude	1.000000	0.138903	-0.047305
Longitude	0.138903	1.000000	0.002554
Quantity	-0.047305	0.002554	1.000000
Unit_Price	0.015004	0.002484	0.080164
Discount	0.067839	-0.049335	-0.105040
Sales	-0.033100	0.006514	0.808042
Profit	-0.042743	0.005054	0.801465

	Profit
Latitude	-0.042743
Longitude	0.005054
Quantity	0.801465
Unit_Price	0.479456
Discount	-0.369269
Sales	0.954188
Profit	1.000000

15. Correlation Heatmap

```
plt.figure(figsize=(10,7))
sns.heatmap(
    numerical_data.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
```



C. EDA Summary

The dataset contains 500 orders. Food generated the highest category sales at ₹16,722.75. Library Café generated the highest outlet sales at ₹13,685.25. Sales have a right-skewed distribution, with a mean of ₹147.09 and a median of ₹120.00. Quantity and Sales show a strong positive relationship, which is also confirmed by the correlation analysis.

9. DESCRIPTIVE STATISTICAL ANALYSIS

Descriptive statistical analysis summarizes the main numerical variables in the Campus Cafe dataset.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("Campus Cafe Dataset.csv")

columns = ["Quantity", "Unit_Price", "Discount", "Sales", "Profit"]

for col in columns:
    print("\n", col)
    print("Mean:", df[col].mean())
    print("Median:", df[col].median())
    print("Mode:", df[col].mode()[0])
    print("Minimum:", df[col].min())
    print("Maximum:", df[col].max())
    print("Range:", df[col].max() - df[col].min())
    print("Variance:", df[col].var())
    print("Standard Deviation:", df[col].std())
```

OUTPUT:

```
Quantity
Mean: 2.688
Median: 2.0
Mode: 2
Minimum: 1
Maximum: 6
Range: 5
Variance: 1.8904368737474952
Standard Deviation: 1.3749315887517806

Unit_Price
Mean: 57.45
Median: 55.0
Mode: 55
Minimum: 25
Maximum: 105
Range: 80
Variance: 405.2580160320641
Standard Deviation: 20.131021236690007

Discount
Mean: 0.0646
Median: 0.05
Mode: 0.0
```

Variable	Mean	Median	Mode	Min	Max	Range	Std. Dev.
Quantity	2.688	2	2	1	6	5	1.375
Unit Price	57.45	55	55	25	105	80	20.131

Discount	0.0646	0.05	0	0	0.15	0.15	0.0565
Sales	147.0905	120	45	21.25	570	548.75	101.562
Profit	45.0912	36.3	36	5.5	171	165.5	31.438

Interpretation:

Sales and Profit have considerable variation because order quantities and product prices differ. The mean Sales value is ₹147.09, while the median is ₹120, indicating that some higher-value orders pull the mean upward.

10. DISTRIBUTION ANALYSIS

To study the distribution of Sales using a histogram and box plot, and understand its shape, central tendency, spread, skewness and possible outliers.

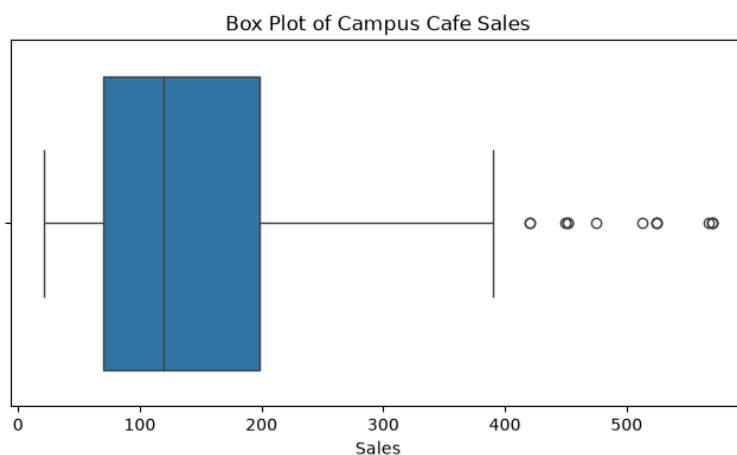
CODE:

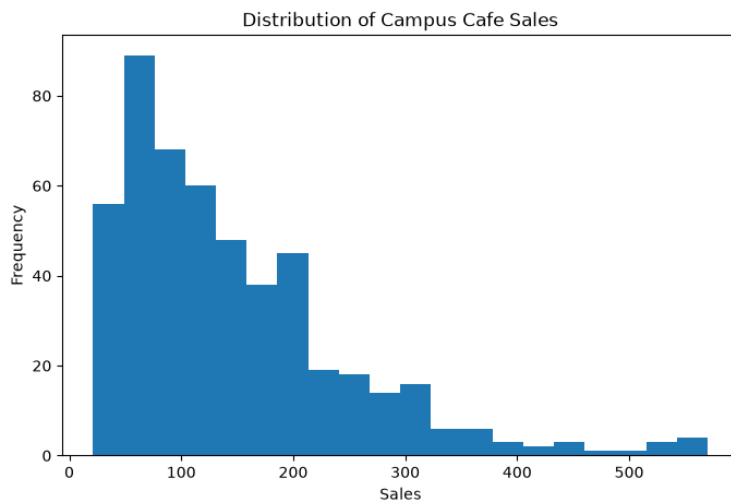
```
plt.figure(figsize=(8,5))
plt.hist(df["Sales"].dropna(), bins=20)
plt.title("Distribution of Campus Cafe Sales")
plt.xlabel("Sales"); plt.ylabel("Frequency")
plt.show()
```

```
plt.figure(figsize=(8,4))
sns.boxplot(x=df["Sales"])
plt.title("Box Plot of Campus Cafe Sales")
plt.xlabel("Sales")
plt.show()
```

```
print("Skewness:", df["Sales"].skew())
```

OUTPUT:





Skewness: 1.5447883805416567

Analysis

The positive skewness indicates a right-skewed Sales distribution. Most orders are concentrated at lower-to-middle sales values, while a smaller number of high-value orders extend the upper tail. The mean Sales of ₹147.09 is higher than the median of ₹120.00.

Short Interpretation

The histogram and box plot show the spread of Campus Cafe sales and help identify unusually high orders. The right-skewed pattern suggests that high-value orders should be considered separately when evaluating typical order performance.

11. CORRELATION ANALYSIS

To study relationships between important numerical variables using a correlation matrix and heatmap.

Numerical Variables Selected

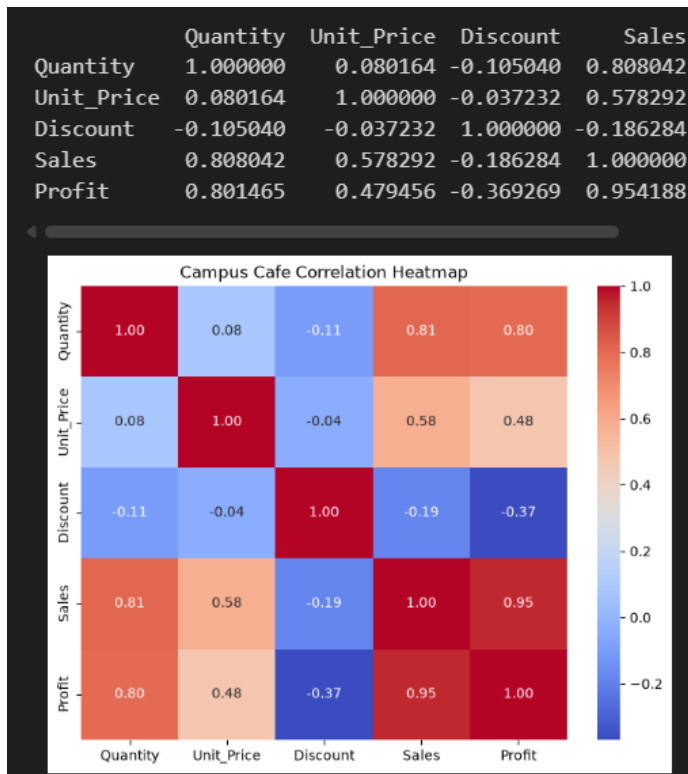
Quantity, Unit_Price, Discount, Sales and Profit.

CODE:

```
data = df[["Quantity", "Unit_Price", "Discount", "Sales", "Profit"]]
correlation = data.corr()
print(correlation)

plt.figure(figsize=(8,6))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Campus Cafe Correlation Heatmap")
plt.show()
```

Output:



Relationship	Correlation
Quantity – Sales	0.808
Quantity – Profit	0.801
Sales – Profit	0.954
Unit Price – Sales	0.578
Discount – Sales	-0.186
Discount – Profit	-0.369

Analysis

Sales and Profit have a very strong positive correlation (0.954). Quantity has a strong positive correlation with Sales (0.808) and Profit (0.801). Unit Price has a moderate positive relationship with Sales (0.578). Discount has a weak negative relationship with Sales (-0.186) and a moderate negative relationship with Profit (-0.369).

Business Interpretation

Higher quantities are strongly associated with higher sales and profit. The strong Sales–Profit relationship indicates that sales performance is closely connected with profitability. The negative Discount–Profit relationship suggests that larger discounts may reduce profit per order, although correlation alone does not establish causation.

Short Conclusion

Correlation analysis identifies the variables most closely associated with sales and profit and provides a basis for regression and business decision-making.

12. REGRESSION ANALYSIS

Independent Variable (X): Quantity. Dependent Variable (Y): Sales. The model checks whether order quantity can linearly explain Campus Cafe sales.

CODE:

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

data = df[["Quantity", "Sales"]].dropna()
X = data[["Quantity"]]
y = data["Sales"]

model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)

print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])
print("R2:", model.score(X, y))
print("MSE:", mean_squared_error(y, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y, y_pred)))

quantity_value = 4
print("Predicted Sales for Quantity 4:",
      model.predict([[quantity_value]])[0])

plt.scatter(X["Quantity"], y)
plt.plot(X["Quantity"], y_pred)
plt.title("Quantity vs Sales Regression")
plt.xlabel("Quantity"); plt.ylabel("Sales")

plt.show()

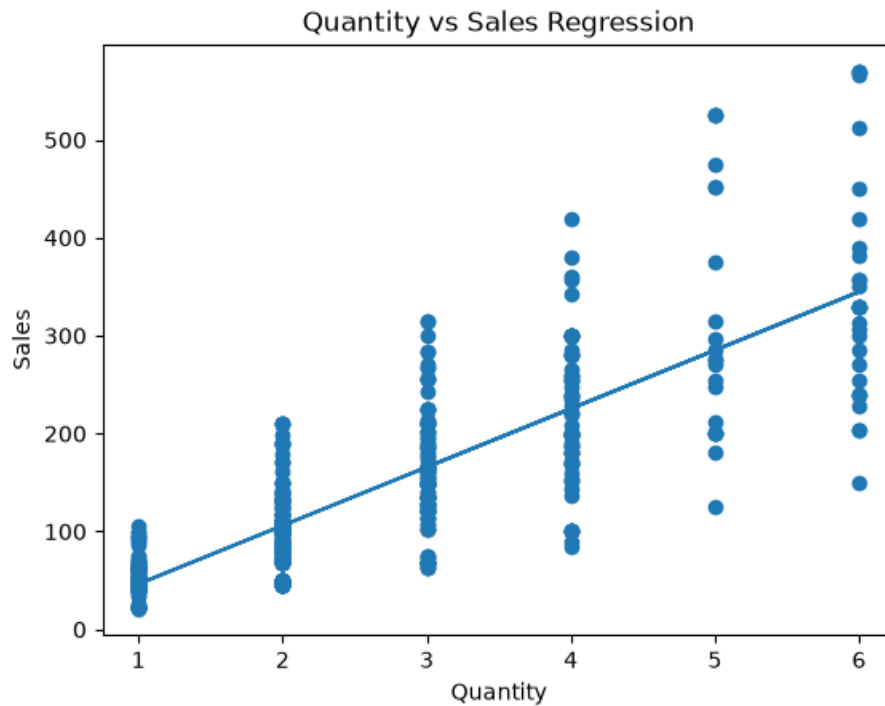
```

Output:

```

Intercept: -13.350149682825247
Coefficient: 59.68774169747962
R2: 0.6529316919755289
MSE: 3572.8125058897863
RMSE: 59.77300817166379
Predicted Sales for Quantity 4: 225.40081710709325

```



Verified result: $\text{Sales} = -13.3501 + 59.6877 \times \text{Quantity}$. $R^2 = 0.6529$. $\text{MSE} = 3572.81$ and $\text{RMSE} = 59.77$. Predicted Sales for an order quantity of 4 is approximately ₹225.40.

Interpretation

The positive regression coefficient means that, on average, an additional unit of quantity is associated with approximately ₹59.69 higher Sales. The R^2 value of 0.6529 indicates that about 65.29% of the variation in Sales is explained by Quantity in this simple linear model.

Business Interpretation

Quantity is a useful predictor of order Sales, although other factors such as product price, category and discount also affect sales.

13. BUSINESS INTELLIGENCE DASHBOARD

To create an interactive dashboard to analyze Campus Cafe sales, profit, products, categories and outlet performance.

Component	Recommended Representation
KPI Card 1	Total Sales – ₹73,545.25
KPI Card 2	Total Profit – ₹22,545.60
KPI Card 3	Total Quantity – 1,344
KPI Card 4	Number of Orders – 500
Bar Chart	Sales by Category
Bar Chart	Profit by Outlet
Pie Chart	Sales Share by Category
Line Chart	Monthly Sales Trend
Map	Outlet/Region Locations
Filters	Category, Product, Outlet, Region, Brand, Date

Titles	Clear title for every chart
--------	-----------------------------

Five Verified Dashboard Insights

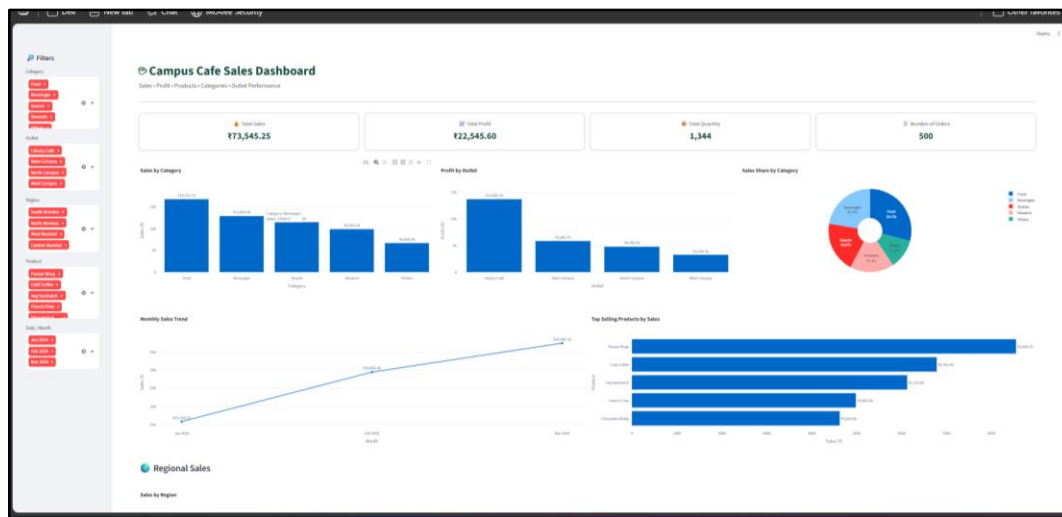
Food is the highest-sales category with ₹16,722.75 in sales.

Library Café has the highest outlet sales at ₹13,685.25.

South Mumbai has the highest regional sales at ₹26,497.00.

Paneer Wrap is the highest-selling product by sales at ₹8,546.25.

Total Campus Cafe sales are ₹73,545.25 and total profit is ₹22,545.60.



14. PROBABILITY AND PROBABILITY DISTRIBUTIONS

To calculate simple probabilities using actual Campus Cafe records and demonstrate Normal, Uniform and Exponential distributions.

Actual Dataset Probability Examples

Probability that a randomly selected order belongs to the Food category = $111/500 = 0.222$ (22.2%). Probability that an order has Quantity ≥ 4 = 0.246 (24.6%). Probability that Sales is above the overall mean = 0.404 (40.4%).

CODE

```
p_food = (df["Category"] == "Food").mean()
p_qty4 = (df["Quantity"] >= 4).mean()
p_sales_mean = (df["Sales"] > df["Sales"].mean()).mean()

print("P(Food) =", p_food)
print("P(Quantity >= 4) =", p_qty4)
print("P(Sales > Mean Sales) =", p_sales_mean)

np.random.seed(10)
normal_data = np.random.normal(df["Sales"].mean(), df["Sales"].std(), 1000)
```

```

uniform_data = np.random.uniform(df["Sales"].min(), df["Sales"].max(), 1000)
exponential_data = np.random.exponential(df["Sales"].mean(), 1000)

for data, title in [(normal_data, "Normal Distribution"),
                    (uniform_data, "Uniform Distribution"),
                    (exponential_data, "Exponential Distribution")]:
    plt.hist(data, bins=30)
    plt.title(title)
    plt.xlabel("Value"); plt.ylabel("Frequency")

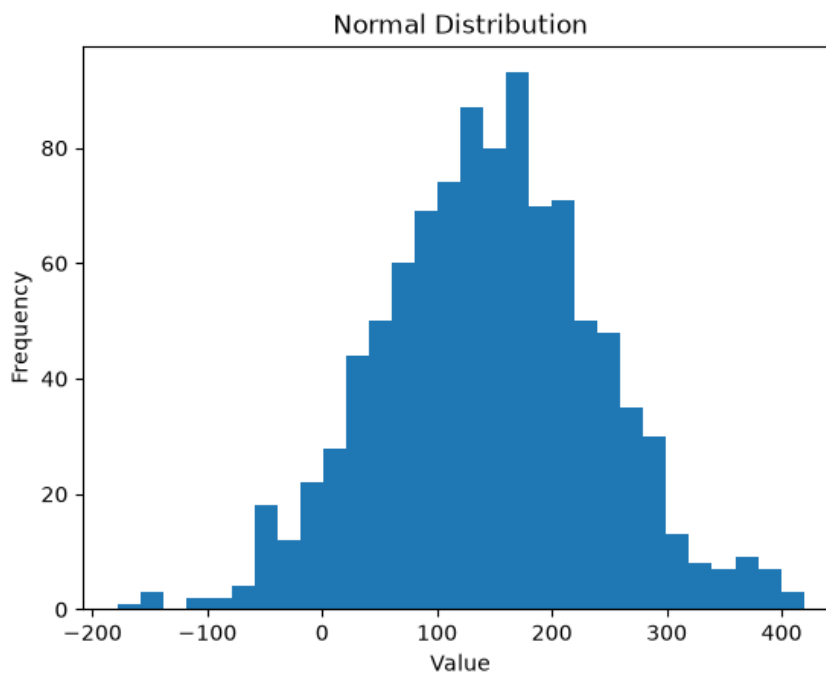
plt.show()

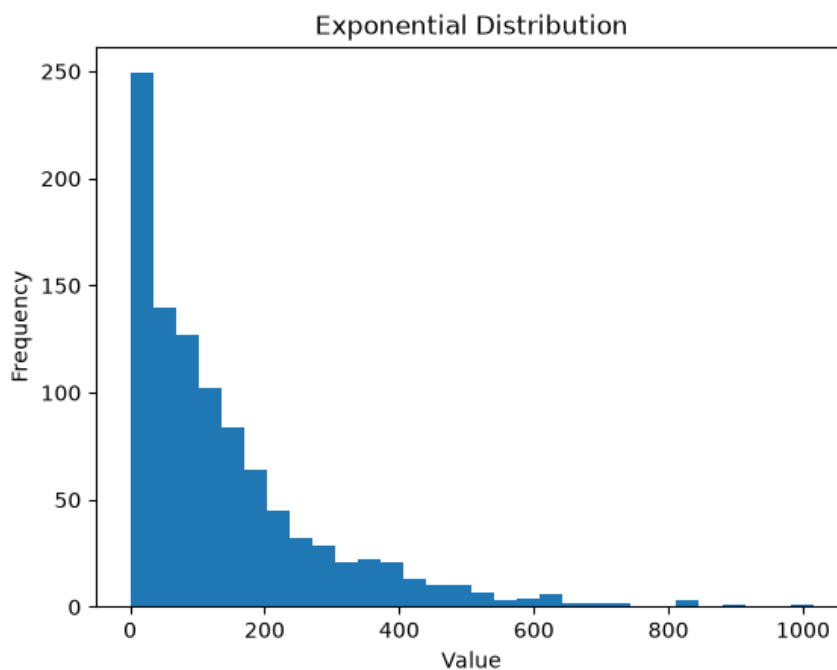
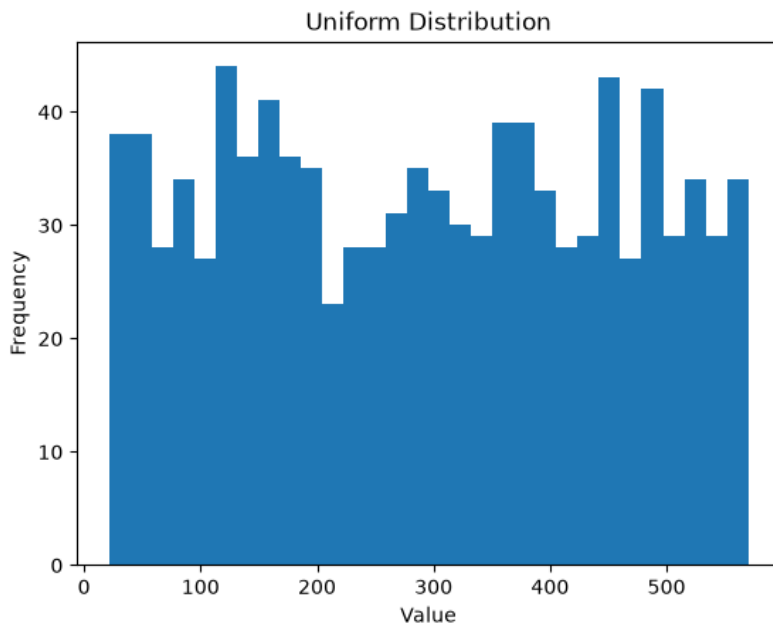
```

```

P(Food) = 0.222
P(Quantity >= 4) = 0.246
P(Sales > Mean Sales) = 0.404

```





Business Interpretation:

Probability can help the cafe estimate the likelihood of demand levels and order characteristics. Normal, Uniform and Exponential distributions provide simple models for variability and uncertainty; the generated distributions are illustrative rather than claims about the empirical shape of every Campus Cafe variable.

15. DECISION MAKING UNDER UNCERTAINTY

To compare inventory alternatives for a campus cafe when future demand is uncertain.

Business Scenario: Campus Cafe must choose between a Low Inventory plan and a High Inventory

plan for a busy day. Based on the observed Quantity data, demand is uncertain. For a simple decision model, assume High Demand has probability 0.40 and Low Demand has probability 0.60. The payoffs are illustrative business assumptions.

Alternative	High Demand (0.40)	Low Demand (0.60)
High Inventory	₹6,000	₹2,000
Low Inventory	₹4,500	₹3,000

```
alternatives = {
    "High Inventory": [(0.40,6000),(0.60,2000)],
    "Low Inventory": [(0.40,4500),(0.60,3000)]
}
for name, outcomes in alternatives.items():
    ev = sum(p*payoff for p,payoff in outcomes)
    print(name, "Expected Value:", ev)
```

```
High Inventory Expected Value: 3600.0
Low Inventory Expected Value: 3600.0
```

Expected Value: High Inventory = ₹3,600. Low Inventory = ₹3,600. Therefore, the two alternatives have equal expected value under these assumed payoffs. The cafe should use additional factors such as stock risk, perishability and customer service before choosing.

Note: The probabilities are informed by the project's uncertainty context, but the payoff table is an explicit illustrative assumption because the CSV does not contain inventory cost/payoff fields.

16. SAMPLING AND SAMPLING DISTRIBUTION

To select random samples of Campus Cafe Sales, calculate sample means repeatedly and study the sampling distribution and Central Limit Theorem.

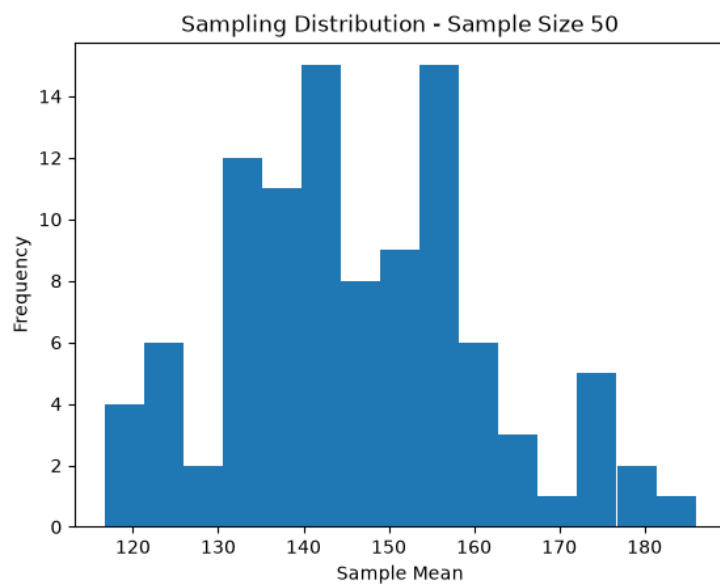
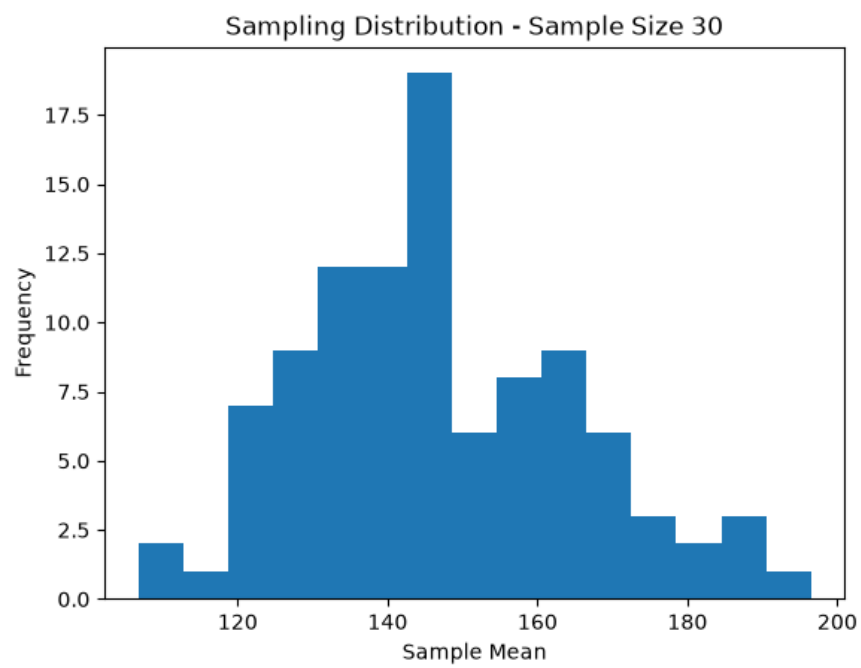
Population: All 500 Sales observations. Sample sizes: 30, 50 and 100. For each sample size, 100 random samples are selected using simple random sampling.

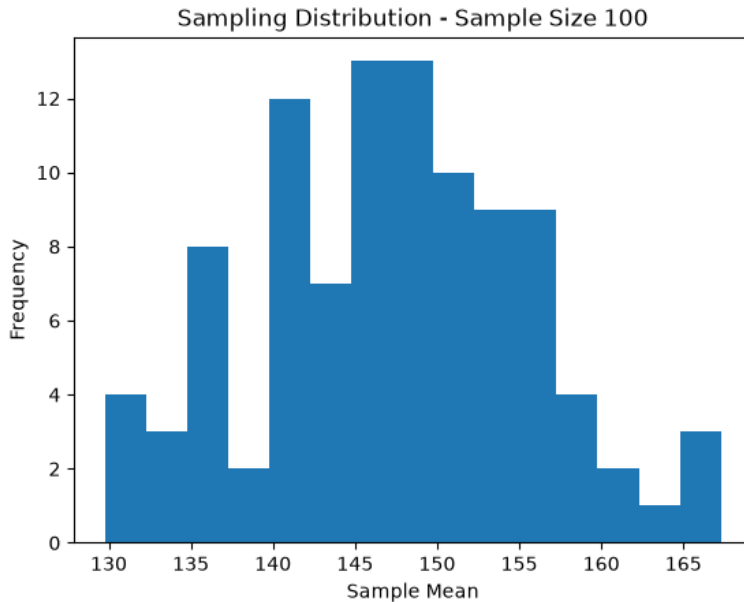
```
population = df["Sales"].dropna().values
np.random.seed(10)

for size in [30,50,100]:
    sample_means = []
    for i in range(100):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means.append(sample.mean())
    sample_means = np.array(sample_means)

    print("Sample Size:", size)
    print("Mean of Sample Means:", sample_means.mean())
    print("SD:", sample_means.std(ddof=1))
    print("Standard Error:",
          population.std(ddof=1)/np.sqrt(size))

plt.hist(sample_means, bins=15)
plt.title("Sampling Distribution - Sample Size " + str(size))
plt.xlabel("Sample Mean"); plt.ylabel("Frequency")
plt.show()
```





Interpretation:

The mean of sample means should be close to the population mean of ₹147.09. As sample size increases, the standard error decreases and the sampling distribution becomes more concentrated, illustrating the Central Limit Theorem.

17. CONFIDENCE INTERVAL ESTIMATION

To estimate the population mean of Campus Cafe Sales using 90%, 95% and 99% confidence intervals.

```
from scipy import stats
sales = df["Sales"].dropna()
n = len(sales)
mean = sales.mean()
std = sales.std()

for confidence in [0.90,0.95,0.99]:
    alpha = 1-confidence
    t_value = stats.t.ppf(1-alpha/2, n-1)
    margin = t_value * std/np.sqrt(n)
    print(confidence, mean-margin, mean+margin)
```

```
0.9 139.60567524605293 154.57532475394706
0.95 138.16669436055176 156.01430563944822
0.99 135.3461640377971 158.83483596220287
```

Confidence Level	Lower Limit	Upper Limit
90%	₹139.61	₹154.58
95%	₹138.17	₹156.01
99%	₹135.35	₹158.83

Interpretation:

The 95% confidence interval estimates that the true average Campus Cafe Sales level is between approximately ₹138.17 and ₹156.01, based on this dataset. The wider 99% interval reflects greater confidence and therefore a wider range.

18. BOOTSTRAPPING

Objective

To estimate a 95% bootstrap confidence interval for the median Campus Cafe Sales using repeated samples with replacement.

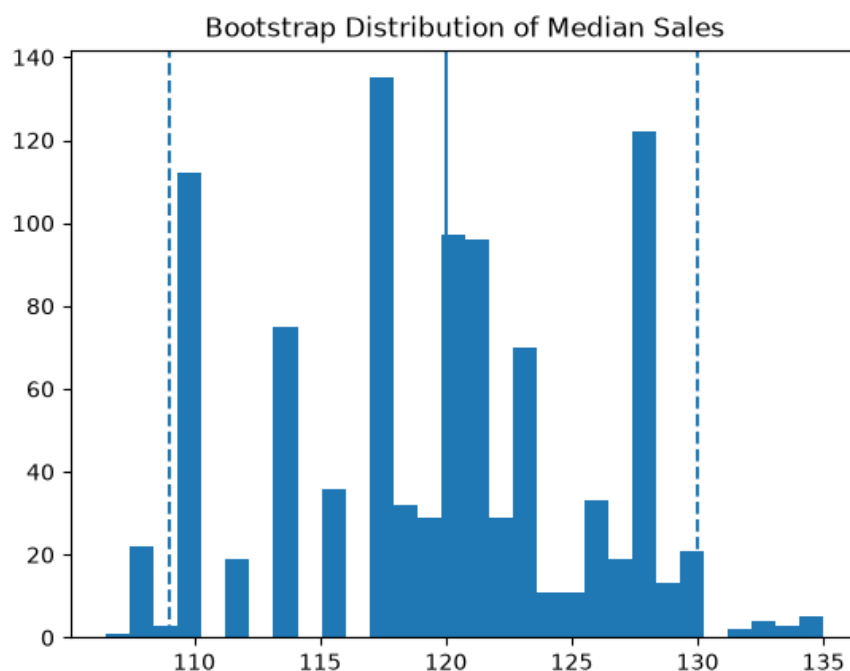
```
sales = df["Sales"].dropna().values
np.random.seed(10)
bootstrap_medians = []

for i in range(1000):
    sample = np.random.choice(sales, size=len(sales), replace=True)
    bootstrap_medians.append(np.median(sample))

bootstrap_medians = np.array(bootstrap_medians)
median = np.median(sales)
lower = np.percentile(bootstrap_medians, 2.5)
upper = np.percentile(bootstrap_medians, 97.5)

print("Original Median:", median)
print("Bootstrap Samples:", 1000)
print("95% CI:", lower, upper)

plt.hist(bootstrap_medians, bins=30)
plt.axvline(lower, linestyle="--")
plt.axvline(upper, linestyle="--")
plt.axvline(median)
plt.title("Bootstrap Distribution of Median Sales")
plt.show()
```



Verified result: Original median = ₹120.00. The 95% bootstrap confidence interval is approximately ₹109.00 to ₹130.00.

Business Interpretation: The bootstrap interval gives a practical estimate of the uncertainty around the typical order Sales value without requiring the Sales variable to be normally distributed.

19. HYPOTHESIS TESTING

Business Question

Is the average Campus Cafe Sales significantly different from ₹150?

Hypotheses

H_0 : Mean Sales = ₹150. H_1 : Mean Sales $\neq$ ₹150. Significance level $\alpha = 0.05$. A one-sample t-test is appropriate.

```
from scipy.stats import ttest_1samp
sales = df["Sales"].dropna()
benchmark = 150

t_stat, p_value = ttest_1samp(sales, benchmark)
print("Test Statistic:", t_stat)
print("p-value:", p_value)

if p_value < 0.05:
    print("Decision: Reject H0")
else:
    print("Decision: Do not reject H0")
```

```
Test Statistic: -0.6405764977454343
p-value: 0.5220919189649235
Decision: Do not reject H0
```

Verified result: $t = -0.6406$ and $p = 0.5221$. Since $p > 0.05$, do not reject H_0 .

Business Interpretation: There is not enough statistical evidence to conclude that the average Campus Cafe Sales differs from the ₹150 benchmark.

20. A/B TESTING

The dataset does not contain a randomized A/B assignment or conversion field. Therefore, a true experimental website A/B test cannot be claimed from the CSV. To satisfy the analytical requirement without inventing conversion data, an A/B-style comparison can be performed between orders with no discount (Group A) and orders with a positive discount (Group B), using Sales as the response. This is observational and does not prove that discounts caused the difference.

Groups

Group A: Discount = 0. Group B: Discount > 0.

```
from scipy.stats import ttest_ind

A = df[df["Discount"] == 0]["Sales"]
B = df[df["Discount"] > 0]["Sales"]
```

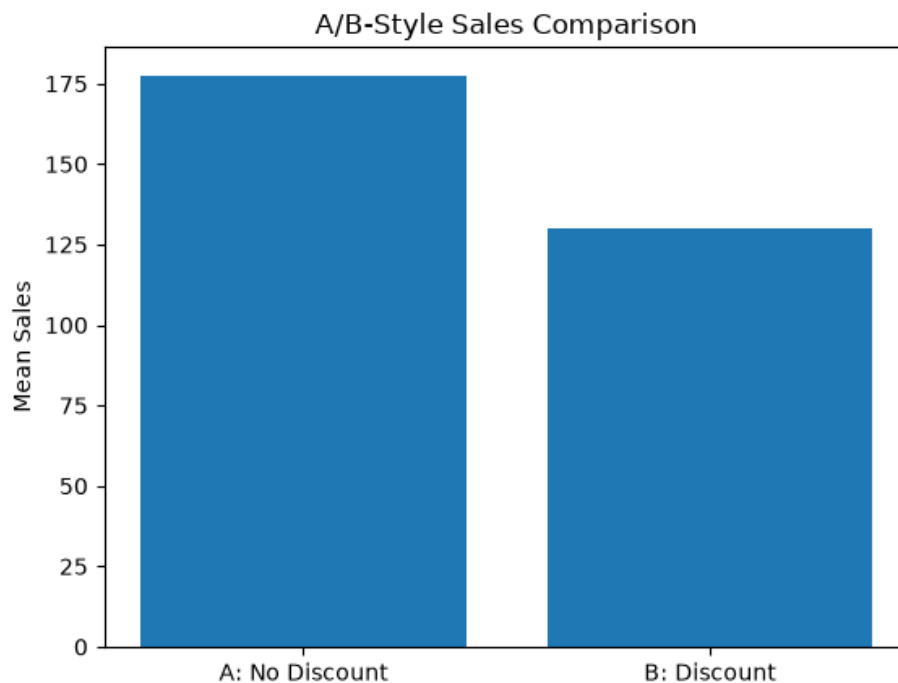
```

print("Group A Orders:", len(A))
print("Group B Orders:", len(B))
print("Group A Mean Sales:", A.mean())
print("Group B Mean Sales:", B.mean())

t, p = ttest_ind(A, B, equal_var=False)
print("t-statistic:", t)
print("p-value:", p)

plt.bar(["A: No Discount", "B: Discount"], [A.mean(), B.mean()])
plt.ylabel("Mean Sales")
plt.title("A/B-Style Sales Comparison")
plt.show()

```



Verified result: Group A has 180 orders with mean Sales ₹177.44. Group B has 320 orders with mean Sales ₹130.02. Welch's t-test gives $t = 4.756$ and $p \approx 0.00000310$, indicating a statistically significant difference in mean Sales between the observed groups.

Business Interpretation: Orders without discounts have higher average Sales in this dataset. However, this is not proof that removing discounts increases sales because customer/product mix may differ between groups. A future randomized A/B experiment would be required for causal conclusions.

21. TIME SERIES ANALYSIS

The dataset contains `Order_Date` from January 2026 through September 2026, so monthly Sales can be analyzed as a time series.

```

import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("Campus Cafe Dataset.csv")

```

```

print("Dataset loaded successfully!")

# Remove extra spaces from column names
df.columns = df.columns.str.strip()

# Convert Order_Date into datetime
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")

print("Date converted successfully!")

# Create monthly sales
monthly = (
    df.groupby(df["Order_Date"].dt.to_period("M"))["Sales"]
    .sum()
    .sort_index()
)

# Convert month to string
monthly.index = monthly.index.astype(str)

# Display monthly sales
print("\nMonthly Sales:")
print(monthly)

# Create chart
plt.figure(figsize=(10, 5))

plt.plot(
    monthly.index,
    monthly.values,
    marker="o",
    linewidth=2
)

plt.title("Monthly Campus Cafe Sales")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.xticks(rotation=45)

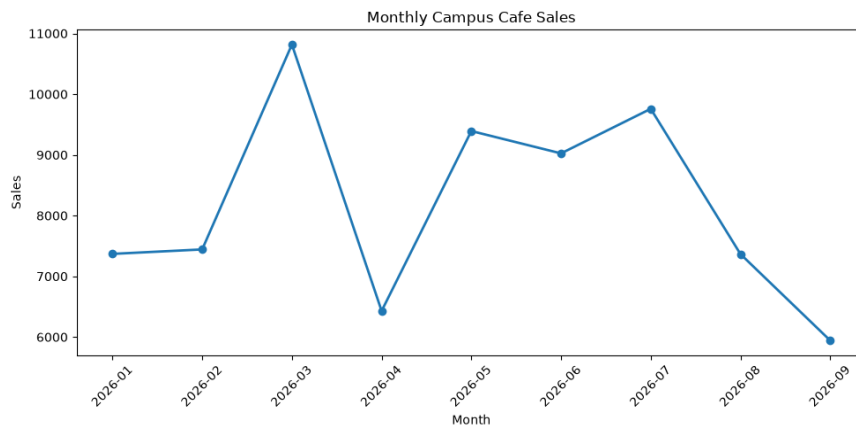
plt.tight_layout()
plt.show()
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA

series = monthly.astype(float)
ets = ExponentialSmoothing(series, trend="add", seasonal=None).fit()
ets_forecast = ets.forecast(3)

arima = ARIMA(series, order=(1,1,0)).fit()
arima_forecast = arima.forecast(3)

print("ETS Forecast:", ets_forecast)
print("ARIMA Forecast:", arima_forecast)

```



Month	Sales
Jan 2026	₹7,369.50
Feb	₹7,443.00
Mar	₹10,817.25
Apr	₹6,429.50
May	₹9,392.75
Jun	₹9,024.75
Jul	₹9,758.75
Aug	₹7,366.25
Sep	₹5,943.50

March 2026 has the highest monthly Sales at ₹10,817.25, while September 2026 has the lowest at ₹5,943.50 in the available period. The dataset contains only nine months, so strong claims about yearly seasonality should be avoided.

Three-month forecasts from the full series are approximately: Exponential Smoothing — Oct ₹7,717.07, Nov ₹7,626.14, Dec ₹7,535.21; ARIMA — Oct ₹6,392.27, Nov ₹6,250.72, Dec ₹6,295.37.

22. FORECAST ACCURACY

To compare forecasting accuracy, the last two observed months (August and September) are kept as test data. The first seven months are used as training data.

```
# Exponential Smoothing
```

```
# =====
```

```
ets = ExponentialSmoothing(
    train,
    trend="add",
    seasonal=None
).fit()
```

```
ets_pred = ets.forecast(2)
```

```
# =====
```

```
# ARIMA
```

```
# =====
```

```
arima = ARIMA(
    train,
    order=(1, 1, 0)
).fit()
```

```

arima_pred = arima.forecast(2)
# =====
# Forecast Accuracy
# =====

for name, pred in [
    ("Exponential Smoothing", ets_pred),
    ("ARIMA", arima_pred)
]:

    mae = mean_absolute_error(
        test,
        pred
    )

    mse = mean_squared_error(
        test,
        pred
    )

    rmse = np.sqrt(mse)

    print("\n", name)
    print("MAE :", round(mae, 2))
    print("MSE :", round(mse, 2))
    print("RMSE:", round(rmse, 2))

```

```

Exponential Smoothing
MAE : 3381.6
MSE : 12192829.82
RMSE: 3491.82

ARIMA
MAE : 2881.27
MSE : 8882927.54
RMSE: 2980.42

```

Model	MAE	MSE	RMSE
Exponential Smoothing	3381.62	12,192,993.94	3491.85
ARIMA	2881.27	8,882,927.54	2980.42

Interpretation: ARIMA has lower MAE, MSE and RMSE on this two-month holdout, so it performs better than the tested Exponential Smoothing model for this particular validation period. Because the dataset has only nine months, this comparison should be treated cautiously.

23. OPTIMIZATION MODELING

Business Problem: Campus Cafe wants to plan production of two high-profit products while considering limited kitchen resources. The actual dataset shows Paneer Rice Bowl has the highest average profit per unit among the listed products at approximately ₹27.34, while Veg Biryani Box averages approximately ₹23.05 profit per unit.

Decision Variables: x = units of Paneer Rice Bowl; y = units of Veg Biryani Box.

Objective Function: Maximize Profit = $27.34x + 23.05y$.

For an illustrative daily planning model, assume two limited resources:
 Ingredient Resource 1: $2x + y \leq 80$; Ingredient Resource 2: $x + 2y \leq 70$; $x, y \geq 0$. These resource limits are assumptions because the CSV does not contain ingredient-resource quantities.

```
from scipy.optimize import linprog
profit = [-27.340385, -23.046296]
```

```
A = [[2,1],
      [1,2]]
b = [80,70]
```

```
result = linprog(
    profit,
    A_ub=A,
    b_ub=b,
    bounds=[(0,None),(0,None)],
    method="highs"
)
```

```
x, y = result.x
max_profit = -result.fun
```

```
print("Paneer Rice Bowl:", x)
print("Veg Biryani Box:", y)
print("Maximum Estimated Profit:", max_profit)
print("Resource 1 Used:", 2*x+y)
print("Resource 2 Used:", x+2*y)
```

```
Paneer Rice Bowl: 30.0
Veg Biryani Box: 20.0
Maximum Estimated Profit: 1281.1374700000001
Resource 1 Used: 80.0
Resource 2 Used: 70.0
```

Verified model result: $x = 30$ Paneer Rice Bowls and $y = 20$ Veg Biryani Boxes, with estimated maximum profit $\approx ₹1,281.14$ under the stated assumptions. Both resource constraints are fully utilized.

Business Interpretation: The model provides a simple production plan when resources are limited. The result is scenario-based because the dataset does not contain actual kitchen resource consumption.

24. SIMULATION MODELING

Business Scenario: Campus Cafe needs to decide how much inventory to keep for daily demand. The observed Quantity values (1–6 units per order) are used as an empirical demand distribution. Three inventory levels—2, 3 and 4 units per order—are compared over 30 simulated demand observations.

```
np.random.seed(10)
demand = np.random.choice(df["Quantity"].values, size=30, replace=True)

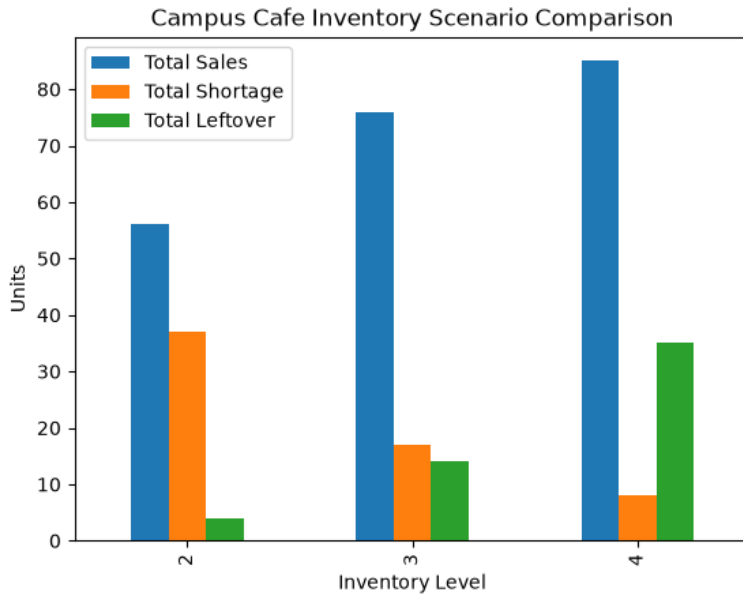
inventory_levels = [2,3,4]
results = []

for inventory in inventory_levels:
    total_sales = np.minimum(inventory,demand).sum()
    shortage = np.maximum(demand-inventory,0).sum()
    leftover = np.maximum(inventory-demand,0).sum()
    service = total_sales/(total_sales+shortage)*100

    results.append([inventory,total_sales,shortage,leftover,service])

results_df = pd.DataFrame(
    results,
    columns=["Inventory Level","Total Sales",
            "Total Shortage","Total Leftover","Service Level (%)"]
)
print(results_df)

results_df.plot(x="Inventory Level",
               y=["Total Sales","Total Shortage","Total Leftover"],
               kind="bar")
plt.title("Campus Cafe Inventory Scenario Comparison")
plt.ylabel("Units")
plt.show()
```



Inventory	Total Sales	Shortage	Leftover	Service Level
2	56	37	4	60.22%
3	76	17	14	81.72%
4	85	8	35	91.40%

Explanation: Increasing inventory improves service level and reduces shortage, but it also increases leftover inventory. In this simulation, inventory level 4 gives the highest service level but also the highest leftover stock.

Business Meaning: The cafe should balance customer service against the risk of excess or perishable inventory.

25. ANOVA

Business Problem: Campus Cafe wants to determine whether average Sales differ among the seven product categories.

H_0 : Mean Sales are equal across all categories. H_1 : At least one category has a different mean Sales.
Significance level $\alpha = 0.05$.

```
from scipy.stats import f_oneway
```

```
groups = [
    group["Sales"].values
    for _, group in df.groupby("Category")
]
```

```
f_statistic, p_value = f_oneway(*groups)
```

```
print("F-statistic:", f_statistic)
print("p-value:", p_value)
```

```
if p_value < 0.05:
    print("Decision: Reject H0")
else:
    print("Decision: Do not reject H0")
```

```
F-statistic: 33.01479562804101
p-value: 1.7903233009526e-33
Decision: Reject H0
```

Verified result: $F = 33.0148$ and $p \approx 1.79 \times 10^{-33}$. Since $p < 0.05$, reject H_0 . There is strong statistical evidence that average Sales are not equal across all product categories.

Business Interpretation: Category choice is associated with substantially different sales performance. The cafe should therefore evaluate category-level demand and profitability separately.

26. EXPERIMENTAL DESIGN

To design an analysis for studying how business factors may affect Campus Cafe Sales.

Component	Description
Factor 1	Order Quantity level
Levels	Low Quantity (1–3) and High Quantity (4–6)
Factor 2	Discount level
Levels	Low Discount (0–0.05) and High Discount (0.10–0.15)
Response Variable	Sales
Experimental Units	Individual order records

How the analysis is conducted: Each order is classified into the defined quantity and discount levels. Mean Sales are compared across the groups. Because the existing data were not generated through random assignment, this is an observational factorial analysis rather than a controlled experiment.

Business Interpretation: The design helps the cafe examine whether order size and discount level are associated with Sales and whether the two factors interact.

27. FACTORIAL EXPERIMENT

To examine the main effects of Quantity level and Discount level and their interaction on Sales using a two-factor analysis.

```
import numpy as np
import statsmodels.api as sm
from statsmodels.formula.api import ols

df["QtyLevel"] = np.where(df["Quantity"] <= 3,
                          "LowQty", "HighQty")
df["DiscLevel"] = np.where(df["Discount"] <= 0.05,
                          "LowDiscount", "HighDiscount")

print(df.groupby(["QtyLevel", "DiscLevel"]).Sales.mean())

model = ols(
    "Sales ~ C(QtyLevel)*C(DiscLevel)",
    data=df
).fit()
```

```
print(sm.stats.anova_lm(model, typ=2))
```

QtyLevel	DiscLevel	
HighQty	HighDiscount	241.133333
	LowDiscount	283.256410
LowQty	HighDiscount	100.523196
	LowDiscount	115.293716
Name: Sales, dtype: float64		
	sum_sq	df
C(QtyLevel)	2.248677e+06	1.0
C(DiscLevel)	5.479400e+04	1.0
C(QtyLevel):C(DiscLevel)	1.638484e+04	1.0
Residual	2.697228e+06	496.0

Effect	F-statistic	p-value	Decision at 0.05
Quantity Level	413.515	2.62e-67	Significant
Discount Level	10.076	0.00160	Significant
Quantity × Discount	3.013	0.0832	Not Significant

Group mean Sales: High Quantity + High Discount $\approx$ ₹241.13; High Quantity + Low Discount $\approx$ ₹283.26; Low Quantity + High Discount $\approx$ ₹100.52; Low Quantity + Low Discount $\approx$ ₹115.29.

Business Interpretation: Quantity level has a very strong association with Sales, and discount level also has a statistically significant main effect in this model. The interaction is not statistically significant at the 5% level. Since this is observational data, the results should be interpreted as associations rather than causal effects.

28. BUSINESS INSIGHTS AND FINDINGS

Total Sales are ₹73,545.25 from 500 recorded orders.

Total Profit is ₹22,545.60, giving an overall profit-to-sales ratio of about 30.66%.

Food is the highest-sales category with ₹16,722.75 sales and ₹5,249.75 profit.

Library Café is the highest-sales outlet with ₹13,685.25 sales.

South Mumbai is the highest-sales region with ₹26,497.00 sales and ₹8,231.30 profit.

Paneer Wrap is the highest-sales product with ₹8,546.25 sales.

Sales and Profit have a very strong positive correlation of 0.954.

Quantity and Sales have a strong positive correlation of 0.808.

Sales has a positive skewness of 1.545, indicating a smaller number of high-value orders.

March 2026 recorded the highest monthly sales at ₹10,817.25, while September recorded ₹5,943.50.

ANOVA shows statistically significant differences in average Sales across product categories ($p < 0.05$).

ARIMA produced lower holdout forecast errors than Exponential Smoothing in the tested two-month validation.

29. BUSINESS RECOMMENDATIONS

Promote high-performing categories such as Food and Meals while maintaining variety in other categories.

Give special attention to high-performing products such as Paneer Wrap and Veg Biryani Box.

Monitor Library Café and South Mumbai performance and study the factors contributing to their stronger sales.

Use quantity-based demand analysis to improve stock planning and reduce shortages.

Monitor discounts carefully because the dataset shows a negative association between Discount and Profit.

Use monthly sales tracking and forecasting to plan inventory and staffing.

Use dashboard filters to compare outlets, products, categories, regions and dates regularly.

Before making pricing or discount decisions, run randomized A/B experiments because the current discount comparison is observational.

30. CONCLUSION

The Campus Cafe Business Analytics project analyzed 500 sales records covering January to September 2026. The analysis examined data quality, sales and profit distributions, relationships between numerical variables, regression, probability, sampling, confidence intervals, bootstrapping, hypothesis testing, A/B-style comparison, time-series forecasting, optimization, simulation, ANOVA and factorial analysis.

The results show that Food is the highest-sales category, Library Café is the highest-sales outlet, South Mumbai is the highest-sales region and Paneer Wrap is the highest-sales product. Sales and Profit have a very strong positive relationship, while Quantity is an important predictor of Sales. Category-level sales differences are statistically significant, and the simulation demonstrates the trade-off between inventory service level and leftover stock.

Overall, the project demonstrates how Business Analytics can convert Campus Cafe transaction data into useful information for sales planning, product management, inventory decisions, forecasting and business decision-making.

31. APPENDIX

The Appendix contains the supporting materials used during the **Campus Cafe Sales Analysis Project**. It includes the Python code, dataset samples, important outputs, statistical analysis results, graphs, forecasting results, simulation results, optimization outputs, and Business Intelligence dashboard screenshots used to support the analysis and conclusions

A. Complete Python Code

Include the Python code used for:

- Data loading and preprocessing
- Exploratory Data Analysis (EDA)
- Descriptive statistics
- Distribution and correlation analysis
- Regression analysis
- Probability analysis

- Sampling and confidence intervals
- Hypothesis testing
- Time-series analysis and forecasting
- ANOVA and simulation

B. Dataset Sample

Include a screenshot of the first 5–10 rows of the Campus Cafe dataset, showing important fields such as:

- Date
- Item/Product
- Category/Item Type
- Quantity
- Price
- Sales/Revenue
- Payment Method

C. Important Outputs

Include screenshots of:

- `df.shape`
- `df.head()`
- Missing-value results
- Duplicate-value results
- Descriptive statistics
- Correlation matrix
- Regression results
- Statistical test results

D. Graphs

Include the important graphs generated during the analysis:

1. Sales by Item Type
2. Top-Selling Items
3. Sales Histogram
4. Sales Box Plot
5. Correlation Heatmap
6. Quantity vs Sales Scatter Plot
7. Monthly Sales Line Chart
8. Forecast Comparison Chart
9. Probability Distribution Graph
10. ANOVA/Simulation Chart

E. Tableau / Power BI Dashboard

Include screenshots of the Campus Cafe dashboard showing:

- Total Sales
- Total Quantity Sold
- Average/Total Revenue

- Sales by Item Type
- Top-Selling Items
- Monthly Sales
- Supplier/Item analysis, if available
- Dashboard filters

F. Decision and Optimization Output

Include the decision tree and optimization results used for the Campus Cafe inventory scenario, showing:

- Decision alternatives
- Demand outcomes
- Probabilities
- Expected values
- Optimal quantity
- Maximum expected profit

G. Simulation Output

Include the simulation results comparing different inventory levels using:

- Total Sales
- Total Shortage
- Total Leftover
- Service Level

H. Statistical Analysis Tables

Include the final tables for:

- Descriptive Statistics
- Correlation
- Regression
- Confidence Interval
- Hypothesis Testing
- ANOVA
- Forecast Accuracy

These appendix materials provide supporting evidence for the analysis, results, and conclusions presented in the Campus Cafe project.